AWS SERVERLESS CUSTOMER DATA PIPELINE

Professional project report | Amazon S3 - AWS Lambda - Amazon DynamoDB

Implementation region: us-east-1 (N. Virginia) Validated: October 23, 2026

Executive summary

Implemented and tested an event-driven, serverless AWS data ingestion application. Uploading a customer JSON file to S3 generates an object-created event that invokes a Python Lambda function. The function retrieves the file, parses its customer records, and writes them to Amazon DynamoDB. The final test resulted in three customer records visible in the Customers table.

Solution architecture

Flow stage	Component / behavior
1. DATA SOURCE	Amazon S3: serverless-customer-data-235494792645
2. EVENT	s3:ObjectCreated:* with .json suffix
3. PROCESSING	AWS Lambda: S3ToDynamoDB (Python 3.13 / boto3)
4. STORAGE	Amazon DynamoDB: Customers (customer_id, String)
5. OBSERVABILITY	CloudWatch logs configured; execution logs not yet reviewed

Resource inventory

Resource	Configured value
Region	us-east-1
S3 bucket	serverless-customer-data-235494792645
Lambda function	S3ToDynamoDB
Runtime	Python 3.13
Execution role	S3LambdaDynamoDBRole
Inline policy	S3LambdaDynamoDBAccess
DynamoDB table	Customers
Partition key	customer_id (String)

Implementation steps

• Created or verified the S3 bucket and DynamoDB Customers table.

• Used the existing Lambda-trusted IAM execution role and AWSLambdaBasicExecutionRole managed policy.

• Corrected the inline IAM policy to permit s3:GetObject on the actual bucket and dynamodb:PutItem on the Customers table.

• Created a Python 3.13 Lambda event function using the existing execution role.

• Deployed Python code using boto3 to read S3 objects, parse JSON, and write DynamoDB items.

• Added the S3 trigger for all object-create events, filtering keys ending in .json.

• Uploaded a test JSON file and verified the DynamoDB table contents.

Security and permissions

The Lambda execution role uses CloudWatch logging permissions through AWSLambdaBasicExecutionRole. The application policy grants S3 object-read permission scoped to the source bucket and DynamoDB item-write permission scoped to the Customers table. The observed policy also included dynamodb:BatchWriteItem, though the implementation uses put_item. No public S3 access is needed.

Validation and troubleshooting

Initial result: DynamoDB Explore items returned 0 items after the first JSON upload. The Lambda Monitor view showed no recent invocation data for the displayed time window; this alone did not prove the function never ran.

Diagnosis: Confirmed the S3 trigger was present and configured with the correct bucket, s3:ObjectCreated:* event type, and .json suffix. Previously corrected the IAM policy to use the actual S3 bucket and Customers table.

Retest: Uploaded customers-test.json as a new object to generate a fresh S3 event. Subsequent DynamoDB inspection showed 3 customer records. The successful end-to-end test is verified by the observed DynamoDB result.

Remaining check: Review CloudWatch log streams for /aws/lambda/S3ToDynamoDB to capture invocation details and verify application log messages.

Confirmed outcome

Metric	Result
Records in Customers table	3
End-to-end processing	Successful on fresh JSON upload
Test file	customers-test.json
CloudWatch execution logs	Not yet independently verified

Operational considerations and enhancements

For production use, add input schema validation, exception handling, retries and dead-letter destinations, idempotency controls, and monitoring alarms. Because put_item writes by partition key, re-uploading the same customer IDs can replace existing records. Avoid committing secrets or real customer data to GitHub. Delete unneeded AWS resources to limit charges.

Skills demonstrated

AWS S3 event notifications; Lambda; Python 3.13; boto3; JSON parsing; DynamoDB NoSQL data modeling; IAM execution roles and resource-scoped policies; serverless architecture; debugging and validation.

Appendix A - Lambda implementation

```
import json
import boto3
from decimal import Decimal
from urllib.parse import unquote_plus

s3 = boto3.client("s3")
dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("Customers")

def lambda_handler(event, context):
    for record in event["Records"]:
        bucket = record["s3"]["bucket"]["name"]
        key = unquote_plus(record["s3"]["object"]["key"])
        print(f"Reading file: {key}")
        response = s3.get_object(Bucket=bucket, Key=key)
        data = json.loads(
            response["Body"].read().decode("utf-8"),
            parse_float=Decimal
        )
        customers = data if isinstance(data, list) else [data]
        for customer in customers:
            table.put_item(Item=customer)
            print(f"Inserted customer: {customer['customer_id']}")
    return {
        "statusCode": 200,
        "body": "Customer records inserted successfully"
    }
```